Reinforcement Learning and the Racetrack Problem - CSCI 446

Connor Munson CONNOR.MUNSON@STUDENT.MONTANA.EDU

Nicholas Kasten NICHOLAS.KASTEN@STUDENT.MONTANA.EDU

Dayton Wickerd DAYTON.WICKERD@STUDENT.MONTANA.EDU

Abstract

This paper explores the performance of three different reinforcement learning algorithms applied to the race track problem. The three algorithms used were Value Iteration, Q-Learning, and State-Action-Reward-State-Action (SARSA). Each algorithm acts as a race car and takes actions within a race track environment based on its structure and flow. Each agent (race car) has a twenty percent chance to fail any action it takes, which introduces a stochastic element to this project. Value iteration is a model-based reinforcement learning algorithm, while Q-Learning and SARSA are different implementations of model-free reinforcement learning algorithms. According to the data, we found that Value Iteration performed better than Q-Learning and SARSA, with Q-Learning and SARSA performing differently depending on the track and crash type. Value Iteration was able to find the most ideal path with a one-hundred percent success rate. Q-Learning and SARSA found less ideal paths with a consistently lower success rate, as they were forced to explore the track and learn from previous runs.

1 Problem Statement and Hypothesis

This project explores different reinforcement learning algorithms and their performance on the race track problem. We used three reinforcement learning algorithms: Value Iteration, Q-Learning, and State-Action-Reward-State-Action (SARSA). Each algorithm will act as a race car, and is tasked with finding an optimal path from the start of a race track to the finish line. We ran these algorithms on 3 types of tracks with different shapes, 2-Track, U-Track, and W-Track. All the tracks were represented in a text file. Each of the tracks each contain a starting line, finish line, and walls the car can bounce off.

Hypothesis: Our initial expectation is that Value Iteration will consistently find the optimal path on all tracks, due to it having full knowledge of the environment. We expect Q-Learning to explore wider paths aggressively and slowly converge. We expect SARSA to be safer with its moves and have fewer collisions, and make less total mistakes. Overall, we expect Value Iteration to perform best in terms of path optimality, while Q-Learning and SARSA will demonstrate trade-offs between exploration efficiency and path safety.

2 Algorithm Description

To solve the racetrack problem, we implemented and evaluated three reinforcement learning algorithms. Value Iteration (VI) is a model-based, dynamic programming algorithm, while Q-Learning (QL) and State-Action-Reward-State-Action (SARSA) are model-free reinforcement learning algorithms.

2.1 Value Iteration

Value Iteration (VI) is a model-based reinforcement learning algorithm used to compute the optimal policy for a Markov Decision Process (MDP). VI works by iteratively updating the value of each state based on the expected rewards from possible actions and the values of successor states. Formally, it applies the Bellman optimality equation:

$$V(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V(s') \right],$$

where $V(s)$ is the value of state s, a represents the possible actions, $P(s' \mid s, a)$ is the transition probability to state s', $R(s, a, s')$ is the reward, and γ is the discount factor.

Our implementation initializes all state values to zero and iteratively updates them until convergence. Once the values stabilize, the optimal policy is extracted by selecting the action with the highest expected value for each state. VI is guaranteed to find the optimal policy for finite MDPs with full knowledge of the transition dynamics. However, its performance can be limited by large state spaces or complex track structures, which increase computational cost.

2.2 Q-Learning

Q-Learning (QL) is an off-policy, model-free reinforcement learning algorithm that estimates the optimal action-value function $Q(s, a)$ without requiring knowledge of transition probabilities. The Q-values represent the expected cumulative reward for taking action a in state s and following the optimal policy thereafter. Q-Learning updates the action-value estimates using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where α is the learning rate, γ is the discount factor, r is the immediate reward, and s' is the resulting state after taking action a.

Our implementation uses an ϵ-greedy exploration strategy to encourage exploration of the state space. Over time, Q-Learning converges to the optimal policy, even in stochastic environments. Although Q-Learning can handle large and complex state spaces more flexibly than Value Iteration, early learning can be unstable and requires many training episodes for convergence.

2.3 SARSA

SARSA (State-Action-Reward-State-Action) is an on-policy, model-free reinforcement learning algorithm that estimates the action-value function while following the current behavior

policy. Unlike Q-Learning, SARSA updates its Q-values using the action the agent actually selects:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right],$$

where a' is the next action chosen by the current policy in state s'.

SARSA also uses an ϵ-greedy exploration strategy, but because updates depend on the policy's chosen actions, it tends to learn safer and more conservative behaviors. In the racetrack environment, this can reduce the likelihood of collisions with walls. Although SARSA may converge more slowly or to slightly less optimal solutions compared to Q-Learning, it generally produces more stable and less risky policies.

3 Experimental Approach

We employed a couple of different methods to describe the performance of our algorithms. First, we ran a grid search on a set of hyper-parameters for each algorithm to find the best parameters to use for each track and crash type. Additionally, we tracked the success rate of simulating the policy multiple times, and the average number of steps each algorithm took to terminate. Then, after finding the best parameters for each algorithm, we analyzed the speed at which each algorithm learned. We wanted to compare each algorithm's performance and the required number of steps for each algorithm to find an ideal path to the finish line.

To gather the learning curve, we had to use two different methods, a method for value iteration, and a method for q-learning and SARSA. For value iteration, we kept track of the deltas (the change in the value funtion) across all iterations. This indicates the rate at which the value function is converging. It is analogous to the learning rate since large deltas indicate large shifts from one iteration to the next, and small deltas indicate small differences in one iteration to the next. For q-learning and SARSA, we opted to analyze the sum of rewards per episode. This roughly indicates learning rate, as the performance of the discovered path is dependent on the reward of a given episode. The two methods of identifying learning rate are not exactly comparable, but they can provide a broader picture as to the ways that these algorithms are similar and different.

4 Results

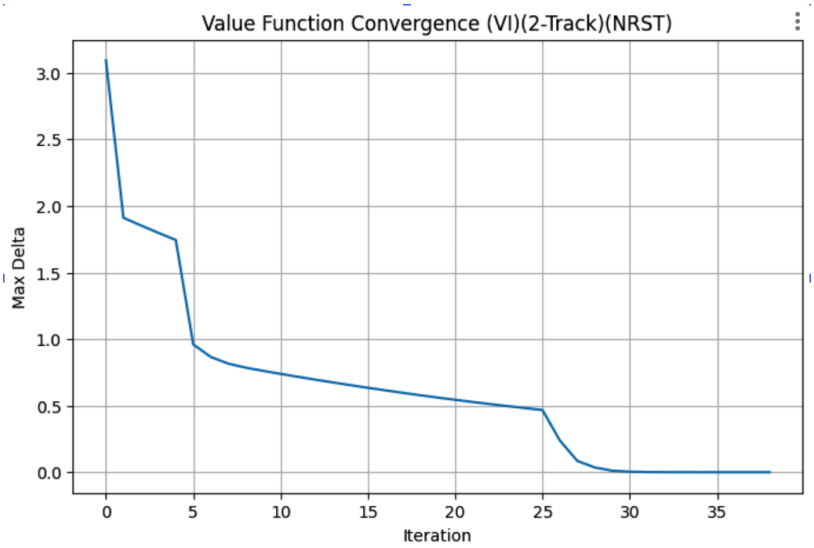

Figure 1: Learning Curve for Value Iteration: $Max\Delta/Iteration$

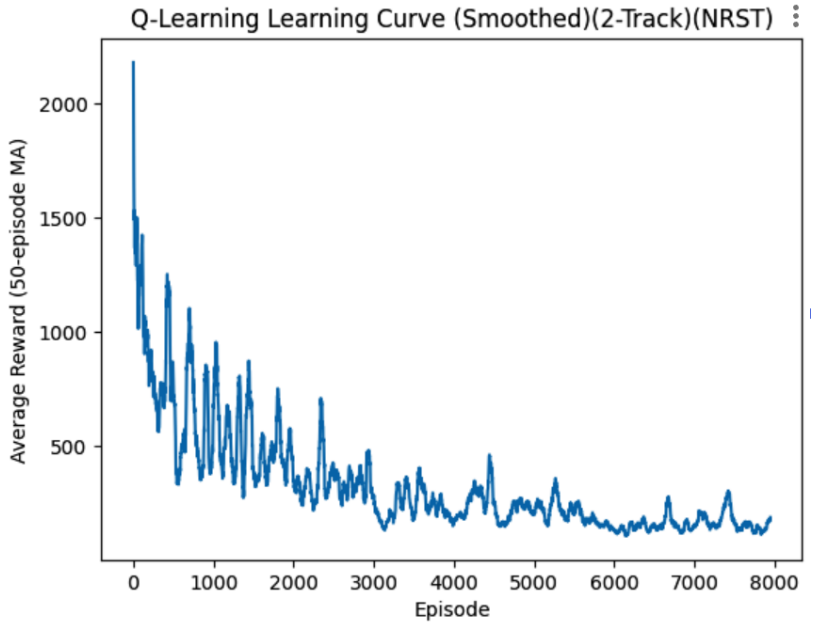

Figure 2: Learning Curve for Q-Learning: *Average Reward/Episode*
Y axis is multiplied by -1 for visual purposes.

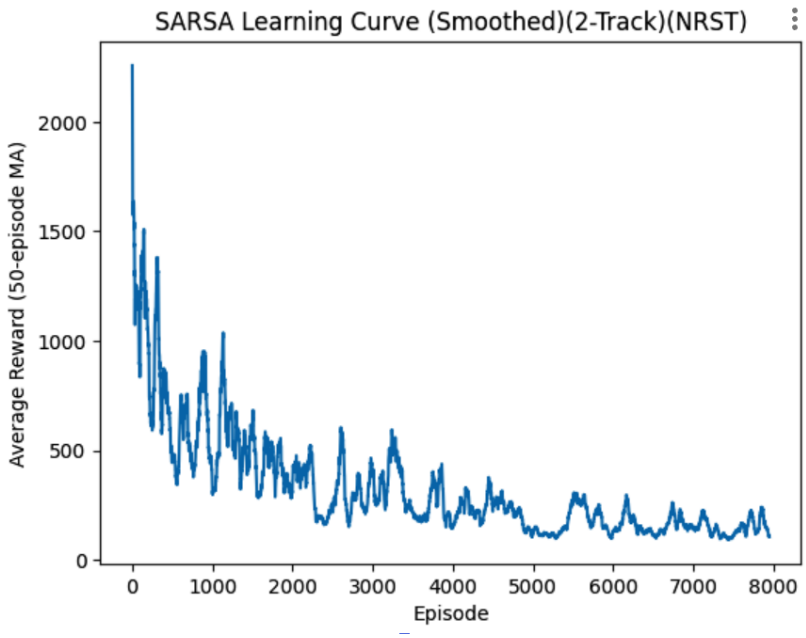

Figure 3: Learning Curve for SARSA: *Average Reward/Episode*
Y axis is multiplied by -1 for visual purposes.

Table 1: Top 5 Hyperparameters and Performance for 2track, Crash Type NSRT

Algorithm	**Gamma**	**Theta**	**Alpha**	**Epsilon**	**Episodes**	**Success Rate**	**Avg Steps**
Q-Learning	0.95	–	0.2	0.05	5000	0.928	484.304
Q-Learning	0.95	–	0.2	0.10	8000	0.918	520.162
Q-Learning	0.95	–	0.2	0.05	8000	0.916	490.918
Q-Learning	0.97	–	0.2	0.10	8000	0.898	595.094
Q-Learning	0.97	–	0.1	0.05	8000	0.894	742.928
SARSA	0.95	–	0.2	0.05	8000	0.992	178.006
SARSA	0.97	–	0.2	0.10	5000	0.918	611.458
SARSA	0.95	–	0.2	0.10	8000	0.902	589.692
SARSA	0.95	–	0.1	0.05	5000	0.898	798.430
SARSA	0.95	–	0.2	0.05	5000	0.872	719.600
Value Iteration	0.99	1×10^{-8}	–	–	–	1.000	26.394
Value Iteration	0.95	1×10^{-4}	–	–	–	1.000	26.498
Value Iteration	0.95	1×10^{-8}	–	–	–	1.000	26.502
Value Iteration	0.97	1×10^{-4}	–	–	–	1.000	26.508
Value Iteration	0.97	1×10^{-6}	–	–	–	1.000	26.530

Table 2: Top 5 Hyperparameters and Performance for 2-track, Crash Type STRT

Algorithm	**Gamma**	**Theta**	**Alpha**	**Epsilon**	**Episodes**	**Success Rate**	**Avg Steps**
Q-Learning	0.97	–	0.2	0.10	8000	1.000	52.402
Q-Learning	0.95	–	0.2	0.10	8000	1.000	55.472
Q-Learning	0.97	–	0.2	0.05	5000	0.994	88.764
Q-Learning	0.97	–	0.1	0.05	8000	0.990	117.966
Q-Learning	0.95	–	0.1	0.10	8000	0.968	220.136
SARSA	0.95	–	0.1	0.10	8000	1.000	79.480
SARSA	0.97	–	0.2	0.05	8000	0.980	210.142
SARSA	0.95	–	0.2	0.10	5000	0.974	317.898
SARSA	0.97	–	0.2	0.10	5000	0.948	401.514
SARSA	0.95	–	0.2	0.05	8000	0.942	443.752
Value Iteration	0.97	1×10^{-4}	–	–	–	1.000	38.054
Value Iteration	0.97	1×10^{-6}	–	–	–	1.000	38.156
Value Iteration	0.99	1×10^{-8}	–	–	–	1.000	38.350
Value Iteration	0.99	1×10^{-6}	–	–	–	1.000	38.448
Value Iteration	0.97	1×10^{-8}	–	–	–	1.000	38.450

Table 3: Top 5 Hyperparameters and Performance for U-Track, Crash Type NRST

Algorithm	**Gamma**	**Theta**	**Alpha**	**Epsilon**	**Episodes**	**Success Rate**	**Avg Steps**
Q-Learning	0.97	–	0.2	0.05	8000	0.996	79.790
Q-Learning	0.97	–	0.2	0.05	5000	0.984	229.324
Q-Learning	0.97	–	0.1	0.10	8000	0.960	304.934
Q-Learning	0.95	–	0.2	0.05	8000	0.948	328.384
Q-Learning	0.95	–	0.2	0.10	5000	0.940	409.936
SARSA	0.95	–	0.1	0.05	8000	1.000	92.964
SARSA	0.95	–	0.1	0.10	8000	1.000	109.272
SARSA	0.97	–	0.1	0.10	5000	0.992	263.860
SARSA	0.97	–	0.2	0.10	5000	0.974	214.270
SARSA	0.95	–	0.1	0.05	5000	0.940	459.422
Value Iteration	0.97	1×10^{-4}	–	–	–	1.000	19.400
Value Iteration	0.95	1×10^{-6}	–	–	–	1.000	19.492
Value Iteration	0.99	1×10^{-8}	–	–	–	1.000	19.580
Value Iteration	0.95	1×10^{-4}	–	–	–	1.000	19.616
Value Iteration	0.95	1×10^{-8}	–	–	–	1.000	19.618

Table 4: Top 5 Hyperparameters and Performance for U-Track, Crash Type STRT

Algorithm	**Gamma**	**Theta**	**Alpha**	**Epsilon**	**Episodes**	**Success Rate**	**Avg Steps**
Q-Learning	0.97	–	0.2	0.10	8000	1.000	35.112
Q-Learning	0.95	–	0.2	0.10	5000	1.000	41.904
Q-Learning	0.95	–	0.2	0.10	8000	0.994	62.942
Q-Learning	0.95	–	0.1	0.10	5000	0.994	78.952
Q-Learning	0.95	–	0.1	0.05	5000	0.992	106.716
SARSA	0.95	–	0.2	0.10	5000	0.988	115.570
SARSA	0.95	–	0.2	0.05	8000	0.988	120.168
SARSA	0.95	–	0.1	0.10	8000	0.986	119.080
SARSA	0.97	–	0.2	0.05	5000	0.984	139.818
SARSA	0.95	–	0.2	0.10	8000	0.978	159.660
Value Iteration	0.99	1×10^{-6}	–	–	–	1.000	24.260
Value Iteration	0.97	1×10^{-8}	–	–	–	1.000	24.402
Value Iteration	0.95	1×10^{-6}	–	–	–	1.000	24.440
Value Iteration	0.95	1×10^{-8}	–	–	–	1.000	24.512
Value Iteration	0.97	1×10^{-4}	–	–	–	1.000	24.514

Table 5: Top 5 Hyperparameters and Performance for W-Track, Crash Type NRST

Algorithm	**Gamma**	**Theta**	**Alpha**	**Epsilon**	**Episodes**	**Success Rate**	**Avg Steps**
Q-Learning	0.97	–	0.2	0.05	8000	0.996	59.138
Q-Learning	0.95	–	0.2	0.10	5000	0.982	122.242
Q-Learning	0.97	–	0.1	0.10	8000	0.974	180.498
Q-Learning	0.97	–	0.1	0.10	5000	0.972	212.430
Q-Learning	0.97	–	0.2	0.10	8000	0.968	197.168
SARSA	0.97	–	0.1	0.10	8000	1.000	76.072
SARSA	0.95	–	0.1	0.10	5000	1.000	83.678
SARSA	0.95	–	0.2	0.10	5000	0.988	111.476
SARSA	0.95	–	0.2	0.05	5000	0.986	108.968
SARSA	0.97	–	0.2	0.05	8000	0.972	170.880
Value Iteration	0.99	1×10^{-4}	–	–	–	1.000	9.958
Value Iteration	0.99	1×10^{-6}	–	–	–	1.000	9.992
Value Iteration	0.95	1×10^{-4}	–	–	–	1.000	10.018
Value Iteration	0.95	1×10^{-8}	–	–	–	1.000	10.018
Value Iteration	0.95	1×10^{-6}	–	–	–	1.000	10.030

Table 6: Top 5 Hyperparameters and Performance for W-Track, Crash Type STRT

Algorithm	**Gamma**	**Theta**	**Alpha**	**Epsilon**	**Episodes**	**Success Rate**	**Avg Steps**
Q-Learning	0.95	–	0.1	0.10	5000	0.988	200.922
Q-Learning	0.95	–	0.2	0.10	8000	0.978	139.482
Q-Learning	0.95	–	0.2	0.05	8000	0.974	160.376
Q-Learning	0.97	–	0.2	0.10	5000	0.952	269.844
Q-Learning	0.97	–	0.2	0.10	8000	0.950	269.828
SARSA	0.97	–	0.2	0.10	5000	1.000	57.162
SARSA	0.95	–	0.1	0.10	5000	0.996	69.802
SARSA	0.95	–	0.1	0.05	5000	0.996	89.764
SARSA	0.97	–	0.2	0.05	8000	0.994	64.754
SARSA	0.97	–	0.1	0.05	8000	0.990	100.308
Value Iteration	0.97	1×10^{-8}	–	–	–	1.000	10.100
Value Iteration	0.99	1×10^{-4}	–	–	–	1.000	10.118
Value Iteration	0.97	1×10^{-4}	–	–	–	1.000	10.134
Value Iteration	0.97	1×10^{-6}	–	–	–	1.000	10.136
Value Iteration	0.95	1×10^{-4}	–	–	–	1.000	10.148

5 Discussion

Across all tracks and crash types, we saw consistent levels of performance for all three algorithms. The biggest differences depended on whether it was a model-based or model-free learning efficiency. Overall, Value Iteration produced the fastest and most reliable solutions across every Track and crash type. Q-Learning and SARSA required significantly more time training to reach similar success rates, and their learning dynamics varied substantially depending on the track complexity and crash rules. On average SARSA and Q-Learning also needed more steps than Value Iteration.

Value Iteration has full access to the model of the track, Value Iteration consistently reached a 100 percent success rate with extremely less average steps than both SARSA and Q-Learning. For example, only 10.1 steps on W-Track (STRT) compared to over 200 steps for Q-Learning under the same conditions. (Table 6). However, Value Iteration does need to complete multiple sweeps over the entire race track. until convergence, which becomes costly on larger or more complex tracks such as W-Track.

Q-Learning and SARSA scaled similarly, since they both rely on sampled interactions rather than full track sweeps. Q-Learning converged faster in deterministic settings but was significantly slower when exploration was high. Q-Learning uses a greedy policy, this leads to reapeatedly overestimating values early on in the process. We can see this outline with visible spikes in the learning curve. (Figure 2). SARSA did something a little different. It maintained a smoother and more stable learning throughout. SARSA is more sensitive to the behavior policy. This resulted in more cautious value estimates that avoided the dramatic overshooting seen in Q-Learning. As a result, SARSA performed better in environments where exploration leads to worse transitions.

Overall, our findings support the initial hypothesis. Value Iteration performed by far the strongest due to the full model of the race tracks being available. Value Iteration achieved the shortest paths and perfect success rates. Q-Learning and SARSA both scale better to large state spaces where a model is unavailable, but they showed long learning times and sensitivity to parameters like track geometry. Q-Learning is more aggressive and can converge faster in some cases, while SARSA is more stable and risk-averse. Together, these differences highlight the core trade-off between model-based and model-free reinforcement learning: VI achieves optimality through computation, whereas Q-Learning and SARSA achieve competence through experience.

6 Summary

This project implemented and compared three reinforcement learning algorithms: Value Iteration, Q-Learning, and State-Action-Reward-State-Action (SARSA) on the racetrack problem. We evaluated their ability to learn efficient paths across three different tracks with different crash scenarios. Additionally, every step taken by these algorithms had a twenty percent chance of failure. As we expected, Value Iteration consistently produced the shortest and most reliable path due to its full knowledge of the transition model. Q-Learning was able to reliably converge to strong policies in our testing but required many episodes to get consistent performance. SARSA tended to lean more conservative, which resulted in high success rates but longer paths. Overall, our results show the tradeoffs between exact

planning and model-free learning, with each algorithm performing better or worse than the others under different conditions.

7 References

References

Sarsa in reinforcement learning. https://builtin.com/machine-learning/sarsa. Accessed: 2025-12-05.

What is a value iteration algorithm in reinforcement learning? https://milvus.io/ai-quick-reference/what-is-a-value-iteration-algorithm-in-reinforcement-learning. Accessed: 2025-12-05.

Value iteration - mastering reinforcement learning. https://gibberblot.github.io/rl-notes/single-agent/value-iteration.html. Accessed: 2025-12-05.

John Amanatides and Andrew Woo. A fast voxel traversal algorithm for ray tracing. http://www.cse.yorku.ca/~amana/research/grid.pdf. Accessed: 2025-12-05.

Eugen Dedu. Bresenham-based supercover line algorithm. https://dedu.fr/projects/bresenham/. Accessed: 2025-12-05.

GeeksforGeeks. Q-learning in reinforcement learning. https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/, Oct 2025. Accessed: 2025-12-05.

Stuart J. Russell, Peter Norvig, and Ming-Wei Chang. *Artificial Intelligence: A Modern Approach*. Pearson, Boston, 2022.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 1998.